SYNTHAGENT: Reliable LLM-Based Synthetic Data Generation for Small Healthcare Datasets

Sancho Amaral Simões[1,*], João P. Vilela[1,2], Pedro Henriques Abreu[1]
[1]*University of Coimbra, CISUC/LASI - Centre for Informatics and Systems of the University of Coimbra, Department of Informatics Engineering*, Coimbra, Portugal
[2]*CRACS/INESCTEC-DCS - FCUP, University of Porto*, Porto, Portugal
[*]Corresponding author
sanchosimoes@student.dei.uc.pt, jvilela@fc.up.pt, pha@dei.uc.pt

***Abstract*—The performance of classifiers trained on small data, common in healthcare, can be improved by adding synthetic records to the training data, provided these do not expose the real patients. However, these synthetic data generation approaches typically do not address the difficulty of small data: they learn only from the few records available. Large language models (LLMs) bring domain knowledge from pre-training, but their output often lacks records or contains records that do not follow the structure of the real data, with missing, extra or misnamed columns and values of the wrong type or outside the expected ranges, which must be detected and corrected before the data can be used. In this context, we developed SYNTHAGENT, an LLM-based approach whose control layer checks and repairs every response against the structure of the real data, and can also enforce value ranges, so that every requested record is delivered with the columns and types of the real data. SYNTHAGENT was tested on ten healthcare datasets with seven LLMs from two providers and compared with thirteen generators. It was the only LLM-based generator that always delivered every requested record in this form, achieved the highest fidelity among the LLM-based generators and the largest median F1-score gain of all, and kept a privacy attack near chance.**

***Index Terms*—small data, synthetic data generation, large language models, healthcare**

I. INTRODUCTION

Many prediction tasks in healthcare, such as diagnosing a disease or anticipating an adverse outcome, have too little data. A rare disease may have only a few dozen diagnosed cases, a single-facility study is limited to the patients it can recruit, and labelling records often requires expert review. Sharing data between facilities could add records, but privacy regulations often restrict it [1], so each facility often has to learn from its own data. We call such datasets *small data*: datasets whose number of records is too low to train a reliable prediction model, a limit that depends on the number of features, the number of classes and the frequency of the outcome [2]. A common remedy for small data, and for a lack of data in general, is synthetic data generation (SDG), which adds to the training set artificial records that follow the distribution of the real ones [3]. This paper focuses on SDG for prediction tasks on small healthcare datasets.

SDG is, however, hardest when data are small. Over-sampling techniques, such as the Synthetic Minority Over-sampling Technique (SMOTE) [4], deep generative models, such as the conditional tabular generative adversarial network (CTGAN) and the tabular variational autoencoder (TVAE) [5], and language models fine-tuned on the target data [6], [7] learn only from the few records they are meant to complement. They can therefore only recombine patterns that already exist in those records, for example by interpolating between two existing patients [4], and most were designed and evaluated on datasets with tens of thousands of records [5], [8], [9]. Large language models (LLMs) can add what the records lack. Pre-trained on large text corpora, they know plausible ranges of healthcare variables and typical relations between them, and they can generate new records from a few real examples placed in the prompt, without any training on the target dataset [6], [10]. This makes them the natural source of synthetic records when data are small.

Their output, however, cannot be used directly as data. An LLM returns text that has to be parsed into records, and a response may contain fewer records than requested, missing or invented columns, or values of the wrong type or outside plausible ranges (Figure 1). Such records cannot be added to a training set as they are. A missing column or a value such as `age="sixty"` stops the training job or becomes a missing value, and a shortfall defeats the purpose of augmentation: the number of synthetic records is usually chosen to balance the classes [4], so a generator that returns 64% of them leaves the minority class under-represented. Like the output of any automatic data source, these records must satisfy integrity constraints before they are loaded [11]–[14]. Approaches that prompt an LLM, such as CLLM [10] and EPIC [15], focus on prompt design and on selecting the generated records, and constrained decoding, offered by LLM providers, fixes the format of each response but not the number of records; none ensures that the requested dataset is delivered. Finally, because the records describe patients, the synthetic data must not reveal the real records used to generate them, which is the reason for generating data instead of sharing the real data.

The problem addressed in this paper is therefore to generate, with an LLM, synthetic records that improve classifiers trained on small healthcare data, delivered as a dataset with the requested number of records and the columns and types of the real data, and without exposing the real patients. Formally, given the training records D of a small dataset, their columns

and types S and the configured number of synthetic records N, a configuration θ of the generator produces a synthetic dataset G_θ. The goal is to find a configuration whose dataset belongs to the set $\mathcal{F}(S, N)$ of datasets with exactly N records that conform to S and that maximizes the gain in classifier performance ΔU, while keeping the distance from the real distribution L_{fid}, the privacy risk L_{priv} and the cost κ low:

$$\theta^\star \in \underset{\theta:\, G_\theta \in \mathcal{F}(S,N)}{\arg\min} \big(-\Delta U,\ L_{\text{fid}},\ L_{\text{priv}},\ \kappa\big)(\theta). \qquad (1)$$

$\mathcal{F}(S, N)$ encodes two integrity constraints, the requested number of records and the columns and types of the real data [16], [17], and the objectives are the utility, fidelity and privacy on which synthetic data are evaluated [3], [18], together with cost: ΔU is the change in classifier performance when G_θ is added to D, L_{fid} the divergence between the real and synthetic distributions, L_{priv} the success of a membership-inference attack, and κ the number of tokens processed; Section V defines how each is measured. The objectives are kept separate rather than combined with weights, because the right trade-off depends on how the data are used; SYNTHAGENT satisfies the constraint by construction and is compared with other generators on the objectives.

We propose SYNTHAGENT (*synthetic data agent*), an SDG system for small healthcare datasets that combines a pre-trained LLM, which proposes new records, with a *control layer*, a deterministic component that checks every LLM response before any of its records enters the synthetic dataset. SYNTHAGENT treats generation as a data-ingestion problem: the control layer selects the real records shown to the model, repairs and validates every response against the columns and types of the real data and asks for corrections, requests exactly the missing records, and records the provenance and cost of every request in an audit trail. Like an LLM agent, it alternates actions (requests to the LLM) with observations (checks of their output), hence its name. The control layer is key: the integrity constraints then hold by construction rather than by trust in the LLM, and each healthcare facility can run the system on its own data and obtain a dataset that can be audited like any other data source.

Our contributions are:

- SYNTHAGENT, an SDG system for small healthcare datasets that requires no training on the target data and whose control layer enforces integrity by construction: it delivers the configured number of records with the columns and types of the real data, with an audit trail of every request and its cost;
- an open-source implementation that works with OpenAI-compatible and Anthropic LLM endpoints, including locally served models.

We ask whether SYNTHAGENT makes LLM-based SDG deliver datasets of assured integrity that are useful for small healthcare data, through four research questions:

RQ1 Does the control layer ensure the integrity of the generated datasets?

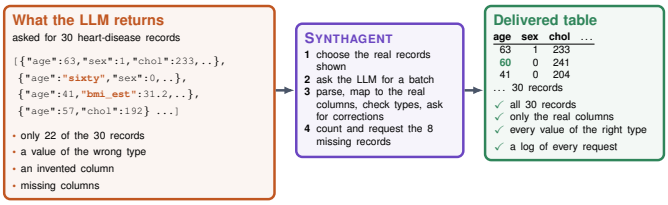


Fig. 1. The problem addressed in this paper, in an illustrative example. Asked for 30 heart-disease records, the LLM (left) returns too few records, an invented column, missing columns and a value of the wrong type. SYNTHAGENT (middle) checks and corrects every response and requests the missing records, following the four steps of Figure 2, and delivers all 30 records with the columns and types of the real data (right).

RQ2 Do the generated records preserve the distribution of the real data without reproducing the real records?

RQ3 Do the generated records improve the performance of classifiers trained on small data?

RQ4 How does SYNTHAGENT compare with established generation strategies, and how much do its results depend on the LLM?

We answer them on ten public healthcare datasets with 116 to 768 records. For each dataset, synthetic records were produced by SYNTHAGENT with commercial LLMs from two providers, by three LLM-based baselines (direct prompting, with and without the provider's option to force responses into a fixed format, and the curation approach of CLLM), and by ten established generators from the oversampling, statistical, tree-based and deep families. Each dataset was split ten times, and on each split ten classifiers were trained with and without synthetic records. The main conclusions follow. Without the control layer, the LLM never returned the configured number of records, even when its responses were forced into the format of the data, whereas SYNTHAGENT always did, with 17% fewer tokens per delivered record (RQ1). The synthetic records reached a fidelity of 72 to 77 out of 100, and a membership-inference attack stayed near chance, although a classifier could still tell synthetic from real records (RQ2). The effect on classifiers depended on the dataset, the classifier and the LLM: the median F1-score gain reached 1.4 percentage points (RQ3, RQ4). SYNTHAGENT gave the largest median gain among sixteen generators; a curation filter improved F1 more often but by less and returned fewer records than configured, and tree-based generators reproduced the distributions more closely at a higher privacy risk (RQ4).

Section II reviews background, Section III describes SYNTHAGENT and Section IV its deployment across healthcare facilities. Section V presents the experimental setup, Section VI reports the results, Section VII discusses implications and limitations, and Section VIII concludes.

II. BACKGROUND KNOWLEDGE

To place the problem of Section I in context, this section reviews the families of tabular SDG methods, their limitations when data are small, and the work on validating generated data and evaluating synthetic data on which SYNTHAGENT builds.

TABLE I

FAMILIES OF TABULAR SDG METHODS: HOW THEY PRODUCE RECORDS, THEIR MAIN LIMITATION IN SMALL DATA, AND THEIR PROPERTIES. ✓: PROVIDED BY DESIGN; ×: NOT PROVIDED. SMOTE'S NUMBER OF RECORDS IS SET BY THE CLASS BALANCE, NOT BY THE USER; FINE-TUNED LANGUAGE MODELS DISCARD UNREADABLE RECORDS BUT DO NOT CHECK TYPES; CONSTRAINED DECODING IS A VARIANT OF PROMPTED LLMS.

Family	How records are produced	Main limitation in small data	No training on target data	Record count	Columns and types checked	Cost recorded
Interpolation and statistical [4], [19]	Combine real records or sample a simple model fitted to them	Only recombine existing patterns; ignore feature meaning	✓	×	×	×
Deep generative [5], [8], [9]	Train a neural network on the real records and sample from it	Few records to fit many parameters; invalid or copied records	×	✓	×	×
Fine-tuned language models [6], [7]	Fine-tune a language model on records written as text	Training on scarce records	×	✓	×	×
Prompted LLMs [10], [15]	Ask a pre-trained LLM for records, with real examples in the prompt	Unreliable output: missing or malformed records, unknown cost	✓	×	×	×
Prompted LLMs, constrained decoding [20], [21]	As above, with the output restricted to a JSON schema	Format guaranteed, but not the number of records; cost unknown	✓	×	✓	×
SYNTHAGENT (ours)	Prompted LLM whose output is checked and completed by a control layer	-	✓	✓	✓	✓

A. Synthetic Data Generation for Tabular Data

Tabular SDG methods fall into four families that differ in how they produce records. *Interpolation and statistical methods* derive new records directly from the real ones or from a simple probabilistic model of them: SMOTE places a new minority-class record on the line segment between two neighbouring records of that class [4], and Bayesian networks estimate the dependencies between features and sample new records from them [19], [22]. *Deep generative models* train a neural network to approximate the joint distribution of the features and sample from it: CTGAN trains a generator against a discriminator, TVAE learns a compressed representation of the records [5], and diffusion models such as TabDDPM, CoDi and TabSyn learn to reverse a gradual corruption of the records [8], [9], [23]; relational and cross-silo variants extend these models to several tables or institutions [24], [25]. *Fine-tuned language models* write each record as a sentence, fine-tune a language model on these sentences and generate new ones, as in GReaT [6], REaLTabFormer [7] and TabLoft [26]. *Prompted LLMs* place a few real records in the prompt of a pre-trained LLM and ask it for new ones, without training, as in CLLM [10] and EPIC [15].

B. Synthetic Data Generation in Small Data

Most of these generators were designed and benchmarked on datasets with tens or hundreds of thousands of records, such as Adult (48,842 records) or Covertype (581,012 records) [5], and their behaviour on a few hundred records is far less studied. Table I summarizes, for each family, how records are produced and its main limitation when data are small. Interpolation and statistical methods, as well as deep generators, are fitted only to the real records, so they cannot add information beyond what those records contain [27, Sec. 2.8]. They also treat every column as an anonymous number, ignoring what is known about variables such as age or cholesterol. Deep generators and fine-tuned language models have many parameters, which a few hundred records cannot estimate reliably, so they may produce records outside the observed ranges and types, or reproduce training records [28]. Prompted LLMs need no training and bring domain knowledge, but nothing ensures that their output contains the configured number of records with the correct columns and types, and the cost of obtaining it is not reported; CLLM [10] filters the generated records by their learning dynamics and EPIC [15] designs balanced prompts, but both concern the quality of the kept records rather than the integrity and cost of the output. SYNTHAGENT keeps the advantages of prompted LLMs and adds the control layer that they lack.

C. Validation of Generated Data

The control layer applies to generation a principle long used in data management: data produced by an automatic process are checked against explicit constraints before they are accepted. Kamino enforces integrity constraints during private data synthesis [11], data-cleaning systems combine statistical inference with integrity constraints [29], [30], and recovering structured records from the free-form output of an LLM has been studied as a data-engineering problem [12], as have LLMs for data wrangling, error detection and document processing, which must cope with unreliable model output [31]–[33]. Constrained decoding restricts the tokens an LLM may emit to a grammar or JSON schema [20], [21], now offered by providers as a structured-output mode; it fixes the format of each response but not the number of records (Section VI-A). In SYNTHAGENT, the constraints are the columns and types inferred from the real data, and every LLM response is checked, in the action-observation loop of LLM agents [34], [35].

D. Evaluating Synthetic Data

Data-preparation studies have shown that a step that looks beneficial can harm downstream models [36], [37], and evaluation frameworks such as Synthcity and SynthEval recom-

Algorithm 1 Generation procedure of SYNTHAGENT

Require: training records D, columns and types S, number of records N, parameters of Table III

1: pool $\mathcal{P} \leftarrow$ random sample of $\rho|D|$ records of D
2: accepted records $A \leftarrow \emptyset$ ▷ empty at the start
3: **repeat**
4: context $C \leftarrow$ next records of $\mathcal{P}$ that fit in a share ω of the context window W ▷ (1)
5: batch $Y \leftarrow$ ask the LLM for $\min(b, N - |A|)$ new records, given S and C ▷ (2)
6: records $X \leftarrow$ parse the batch Y, repairing it if needed, and map its columns to S ▷ (3)
7: correct the invalid values of X, for at most c rounds ▷ (3)
8: $A \leftarrow A \cup X$; log tokens, repairs and violations ▷ (4)
9: **until** $|A| \geq N$ **or** $t_{\max}$ rounds **or** $u_{\max}$ empty answers
10: **return** synthetic table G, the first N records of A, and the audit trail

mend assessing fidelity, privacy and utility together [3], [18]. Fidelity can be complemented by a classifier two-sample test, which asks whether a classifier can tell real from synthetic records [38], and privacy by membership-inference attacks, which ask whether an attacker can tell which records were used for generation [28], [39]. Formal guarantees require models such as ℓ-diversity, t-closeness [40], [41] or differential privacy [42], which SYNTHAGENT does not claim.

III. THE SYNTHAGENT SYSTEM

Section II showed that no family both brings knowledge beyond the few real records and guarantees a usable dataset. SYNTHAGENT does both. It meets the constraint of Equation (1), a synthetic dataset with exactly N records that conform to S, by construction rather than by trusting the LLM. This guarantee is the core of the system: the LLM only proposes records; the control layer decides which to accept, asks for corrections or more records, and stops only when N conforming records have been accepted or a stopping limit is reached. The objectives of Equation (1) are left to the LLM's domain knowledge and the real records shown to it; they are measured, not optimized, in Section VI. The real dataset is never modified.

Figure 2 shows the architecture and Algorithm 1 the procedure. A profiling step first infers the columns and their types S from the training records. Each request then passes through four stages: the context budgeter selects the real records shown to the LLM (1), the LLM proposes a batch of records (2), the deterministic verifier parses, maps and validates it (3), and the admitted records are counted against N (4). The run produces two outputs: the synthetic table G and an audit trail of every request. The rest of this section follows this order and ends with the parameters and the reason for each value.

A. Controller State

The four stages share one state, the register at the bottom of Figure 2, which each stage reads and updates. At every step t, the control layer keeps

$$z_t = (S,\, C_t,\, A_t,\, N - |A_t|,\, B_t,\, E_t), \tag{2}$$

made of the columns and types S, the real records C_t currently shown to the LLM, the synthetic records A_t accepted so far, the number of records still missing $N - |A_t|$, the number of tokens B_t still available in the context window, and the log E_t of parser and validator results. From this state the LLM, denoted π_ϕ, proposes a batch $Y_t \sim \pi_\phi(\cdot \mid z_t)$, and the accepted set grows only through the verifier:

$$A_{t+1} = A_t \cup K\big(V_S\big(P_S(Y_t)\big)\big), \tag{3}$$

where P_S parses the batch Y_t and maps its columns to S, V_S validates the types, and K returns the records after sending their invalid values back to the LLM for correction, in at most c rounds. Because the state is explicit rather than implied by the conversation history, the number of missing records is always known, and the parser log E_t records every failure for later inspection.

B. Context Budgeter (1)

Each request starts with the context budgeter (stage 1; line 4 of Algorithm 1), which decides which real records the LLM sees and sets C_t and B_t in the state. It draws a random sample of $\rho|D|$ training records, between 31 and 123 records in our datasets, written as compact JSON with the feature columns and the class, so that the model sees how features and class co-occur. Because a long prompt can exceed the context window W of the LLM, each request shows as many records of the sample as fit in a share ω of the window:

$$\begin{aligned} C_t &= \mathrm{Slice}_t\big(\mathrm{Select}_s(D, \lceil \rho|D| \rceil), k_t\big), \\ k_t &= \max\{k : T(C_{t,k}) \leq \omega W\}, \end{aligned} \tag{4}$$

where Select_s draws the sample with strategy s, $C_{t,k}$ are the first k records of that slice, T estimates the length in tokens of a prompt with them, and Slice_t rotates through the sample, so that successive requests see different patients. If the provider still rejects a request as too long, the budgeter shows fewer records and retries. Besides the default random strategy, *dense* prefers records with fewer missing values, and *diverse* selects the records closest to the centres of k-means clusters. Neither strategy is class-stratified, a limitation discussed in Section VII.

C. LLM Proposal and Prompts (2)

With these records, the LLM is asked for a batch Y_t (stage 2; line 5). It receives the same three-part conversation for every dataset and LLM (Table II). A first message gives the column names and the sampled records in a compact format, a list of column names followed by one list of values per record. A generation message then asks for an exact number of new records, at most b, and states an output contract: JSON only, the same columns in the same order, a value in every cell, raw numbers, and no additional columns. No free-text description of the dataset or of the prediction task is added, so that no dataset receives more prompt engineering than another. The prompts are therefore deliberately simple, and their wording is not part of the contribution: the guarantees come from the stages that check the response.

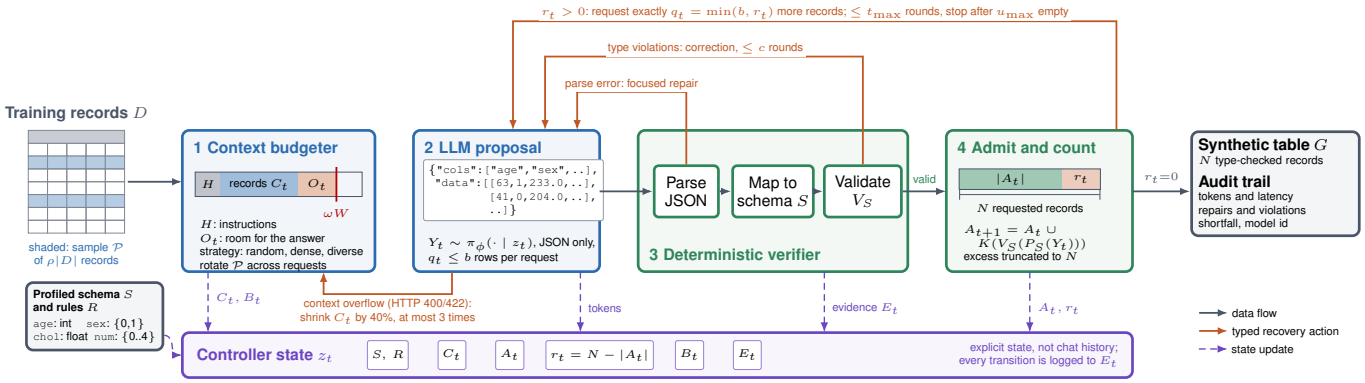


Fig. 2. Architecture of SYNTHAGENT. The context budgeter selects the real records shown to the LLM (1); the LLM proposes a batch of records (2); the deterministic verifier parses it, maps it to the columns and types S and validates it (3); and the admitted records are counted against the configured number N (4). Each failure type triggers its own correction (orange). Every stage reads or updates the controller state z_t (purple). The outputs are the synthetic table G and the audit trail of the run.

TABLE II
WHAT THE LLM RECEIVES IN ONE GENERATION REQUEST. THE PROMPTS ARE THE SAME FOR EVERY DATASET AND LLM. "NOT SUPPLIED" IS INTENTIONAL: THOSE SIGNALS CANNOT INFLUENCE THE GENERATED RECORDS.

Context layer	What the model receives	Why it is included or excluded
Schema	Ordered column names and an exact column-count requirement	Fixes the record interface and prevents invented or omitted fields.
Demonstrations	Compact row-major serialization of a sample of $\rho\|D\|$ training records, with feature and target values	Exposes joint patterns, value formats, label prevalence and plausible cross-column combinations without sending the whole local table.
Generation task	Exact number of new records for the current batch, at most b	Turns an open-ended completion into a bounded action whose output count can be verified.
Output contract	JSON only, fixed column order, complete cells, raw numeric values, no extra metadata	Makes parsing deterministic and removes interface failures unrelated to statistical quality.
Dynamic feedback	Parser error location after malformed JSON, or grouped field violations after failed validation	Reveals only the evidence needed for a targeted repair instead of replaying the full history.
Free-text metadata	Not supplied	Prevents unequal, dataset-specific prompt engineering from confounding the comparison.
Test information	No holdout rows, predictions, metrics or classifier identities	Preserves the evaluation boundary and prevents optimization against the test set.
Global statistics	No separately computed means, quantiles, correlations or class quotas	Fidelity must arise from the selected records; the consequence is weaker protection against rare-mode omission.

D. Deterministic Verifier (3)

The batch Y_t is text, not yet data. The verifier (stage 3; lines 6-7) turns it into records that conform to S, in three steps, and writes every failure to E_t. *Parsing* extracts the records with a local parser that removes surrounding text and code fences, deletes trailing commas, inserts missing commas between records and closes a response truncated by the output limit. If parsing still fails, the control layer sends one repair request that quotes the position and the text around the error, at a low temperature so that the repair fixes the syntax without changing the content. *Mapping* assigns the returned values to the columns of S: by position when the response has the expected number of columns, and otherwise by matching the returned column names to those of S, ignoring case and spacing; an invented column is dropped and a missing one is left empty. *Validation* checks every value against the type that profiling inferred for its column (integer, real or categorical); optional rules can also bound numerical ranges or restrict categories, but they were off in our experiments except in the range-enforcement variant. The invalid values are grouped by column and sent back to the LLM with one example per column, for at most c correction rounds, so that the model corrects only what is wrong instead of regenerating the batch. The three steps escalate from cheap to costly: a defect that the local parser or the mapping can fix costs no request, a syntax error costs one repair request, and only type violations cost correction rounds. In the example of Figure 1, the parser recovers the 22 records, the mapping drops the invented column `bmi_est` and leaves the missing columns empty, the validator flags `age="sixty"`, and one correction round replaces it with a number; the completion loop then requests the eight missing records. Every step is written to the parser log E_t.

E. Admit and Count (4)

Only the records that pass the verifier reach the last stage (stage 4; line 8), which adds them to A_t and decides whether to ask for more. LLMs often return fewer records than requested, so the admitted records are counted against N, and after the

TABLE III
PARAMETERS OF THE CONTROL LAYER, THE VALUES USED IN THE EXPERIMENTS AND THE REASON FOR EACH VALUE. THE LLM-CHOSEN SETTINGS VARIANT (SECTION V) LETS THE LLM CHOOSE THE FIRST AND FOURTH INSTEAD.

Parameter	Value	Reason
Context sample ρ	20% of training records	31 to 123 records in our datasets: enough to show the main patterns of the data, while the prompt still fits the context window of every LLM tested
Window share ω	85%	Leaves room for the answer and for errors in the estimated prompt length
Batch size b	30 records	The longest answer (30 records of the 30-feature dataset) stays within the output limit of every LLM tested; a failed answer loses at most 30 records
Temperature	0.3	Favours adherence to the output format while keeping some diversity
Repair temperature	0.1	A repair should fix the syntax, not change the content
Correction rounds c	2	Bounds the cost of a batch that keeps failing
Top-up rounds $t_{\max}$	50	$50 \times 30 = 1{,}500$ records, more than twice the largest N (614)
Empty-answer limit $u_{\max}$	5	Five consecutive empty answers indicate a provider failure rather than a sampling accident

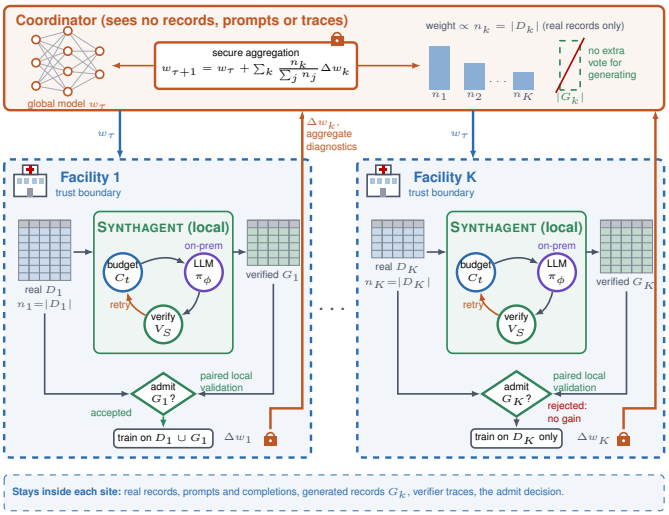


Fig. 3. Deployment across healthcare facilities. Each facility runs SYNTHAGENT (context selection, a locally served LLM, validation and correction) inside its own infrastructure, then decides with paired local validation whether to use the synthetic records G_k (Facility 1) or to train on real data only (Facility K). Only the model update Δw_k and approved aggregate diagnostics leave the facility. The coordinator weights each update by the number of real records $n_k = |D_k|$ (Equation (7)), so generating more records gives a facility no extra weight.

planned batches the completion loop requests only the missing records,

$$r_t = \max(0, N - |A_t|), \qquad q_t = \min(b, r_t), \tag{5}$$

where r_t is the number of records still missing and q_t the number requested in round t, at most the batch size b; the loop also truncates any excess. It stops when N records have been accepted, after $t_{\max}$ rounds, or after $u_{\max}$ consecutive empty answers; a run that ends short because the provider keeps failing is recorded as such. The run returns the synthetic table G and an audit trail with the tokens processed, the generation time, the repairs, the violations and the requested and delivered numbers of records, which makes a synthetic dataset inspectable after the fact and gives the cost κ in Equation (1) a measured value.

F. Parameters

The parameters introduced with each stage are collected in Table III with the values used in the experiments, fixed beforehand from the size of our datasets and the context and output limits of the LLMs rather than tuned on the results; Section VI-A examines their effect.

IV. DEPLOYMENT ACROSS HEALTHCARE FACILITIES

Section III described SYNTHAGENT at one facility. Healthcare facilities, however, often cannot pool their records (Section I) [1], [43], so this section outlines how SYNTHAGENT can serve several of them, a design this paper does not evaluate. Because the control layer needs only a local dataset, its schema and an inference endpoint, each facility of a network can run it inside its own infrastructure. Consider K healthcare facilities, each holding a small dataset D_k for the same prediction task, with different case mix, coding practice and class prevalence. Each facility runs SYNTHAGENT locally and trains on its real and synthetic records,

$$G_k = \mathcal{A}(D_k, S_k, N_k), \qquad \Delta w_k = \mathcal{U}(w_\tau, D_k \cup G_k), \tag{6}$$

where $\mathcal{A}$ denotes SYNTHAGENT, S_k are the local columns and types, N_k the local number of synthetic records, w_τ the shared model at aggregation round τ and $\mathcal{U}$ the local training procedure. Only the model update Δw_k and approved aggregate diagnostics leave the facility. A coordinator aggregates the updates as in FedAvg [44], [45],

$$w_{\tau+1} = w_\tau + \sum_{k \in \mathcal{K}_\tau} \frac{n_k}{\sum_{j \in \mathcal{K}_\tau} n_j} \Delta w_k, \qquad n_k = |D_k|, \tag{7}$$

where $\mathcal{K}_\tau$ is the set of facilities taking part in round τ, preferably under secure aggregation [46]. The weights count *real* records, $|D_k|$ rather than $|D_k \cup G_k|$, so a facility gains no extra weight by generating more records (Figure 3). Unlike SiloFuse, which trains a shared generator across sites [25], this design needs no generator training across facilities, and each facility can reject its synthetic records when paired local validation shows no gain. The design is not, by itself, a privacy guarantee: with a remote LLM the prompt content leaves the facility, so sensitive deployments need a locally served model or a data-processing agreement, and both synthetic records and model updates can leak information [47]. Our experiments evaluate the generation step one facility at a time, as the next section describes; a multi-facility evaluation is future work.

V. EXPERIMENTAL SETUP

Having described the system, we now evaluate it on the four research questions of Section I. This section describes the data,

TABLE IV

EVALUATION DATASETS. n: RECORDS; d: FEATURES; k: CLASSES; $n_{\min}$: RECORDS IN THE SMALLEST CLASS; $n^\star$: MINIMUM SAMPLE SIZE BY THE FIRST CRITERION OF RILEY ET AL. [2] (SHRINKAGE 0.9, $d(k-1)$ PARAMETERS, ANTICIPATED R^2_{CS} OF 15% OF ITS MAXIMUM); RATIO: $n^\star/n_{\mathrm{tr}}$, WITH $n_{\mathrm{tr}} = 0.8n$ THE TRAINING SIZE.

Dataset	n	d	k	$n_{\min}$ (%)	$n^\star$	Ratio
Heart disease	303	13	5	13 (4.3)	10,201	42.1
Breast cancer	569	30	2	212 (37.3)	2,302	5.1
Heart failure	299	12	2	96 (32.1)	946	4.0
Diabetes	768	8	2	268 (34.9)	621	1.0
Liver disease	583	10	2	167 (28.6)	808	1.7
Parkinson's	195	22	2	48 (24.6)	1,851	11.9
Breast cancer (C.)	116	9	2	52 (44.8)	677	7.3
Hepatitis	155	19	2	32 (20.6)	1,688	13.6
Kidney disease	400	24	2	150 (37.5)	1,840	5.8
Thoracic surgery	470	16	2	70 (14.9)	1,606	4.3

For heart disease, $n^\star$ applies the binary criterion to the smallest class versus the rest and is an approximation. Parkinson's contains several recordings per subject, so its effective number of independent records is lower and its ratio higher than shown.

protocol, configurations and measures used to answer RQ1 to RQ4.

A. Datasets

We use ten public classification datasets (Table IV): six widely used benchmarks, five from the UCI repository [48]–[52] and the Pima Indians diabetes data [53], and four further UCI datasets that are smaller, have missing values or mixed types, or are imbalanced [54]–[57]. Each has fewer training records than the minimum sample size recommended for developing a prediction model for its task by the criteria of Riley et al. [2]. Together they span the factors that make small data a challenge for the classifier: 116 to 768 records, 8 to 30 features, two to five classes, a smallest class from 4.3% to 44.8% of the records and up to 10.1% missing values. We keep the original five-class heart-disease target, code the targets of the four further datasets as 0/1 and their binary categories as 0/1, and impute missing values with the mean for numerical and the mode for categorical features, separately within each training and test split. The control-layer variants, the LLM variants and the sensitivity study use the six original datasets; the main configurations and all established generators use all ten.

B. Protocol

Measuring whether synthetic records help a classifier requires real records that the generator has never seen. Each dataset is therefore divided into a training set, given to the generator, and a test set of 20% of the records, used only to score classifiers. Because a test set of 24 to 154 records makes a single division unreliable, the division is repeated ten times with different random seeds, and means and standard deviations are reported. In each repetition, the number of synthetic records N equals the number of training records, so augmentation doubles the training set; because records are requested in batches of at most $b = 30$, a generator that falls short does so on small requests, not because N is large. Ten classifiers covering the main model families are trained: logistic regression (LR), support vector machine (SVM), k-nearest neighbours (kNN), Gaussian naive Bayes (NB), decision tree (DT), random forest (RF), gradient boosting (GB), AdaBoost, XGBoost and a multilayer perceptron (MLP), with library defaults except the number of neighbours, tree depth and number of estimators, which scale with the training size. Using several families avoids conclusions tied to one learner, since interpolated records can help a nearest-neighbour classifier and harm a boosted ensemble. Each classifier is trained in three setups, (1) on real records only, (2) on synthetic records only and (3) on real plus synthetic records, and all three are scored on the same test set. XGBoost failed on liver disease in every repetition because it requires classes coded from 0, whereas this dataset codes them as $\{1, 2\}$, so 99 of the 100 classifier-dataset combinations are reported.

C. Configurations

Table V lists the 36 configurations. A configuration is included in the analysis only if it completed all its datasets and ten repetitions. The *reference* configuration is SYNTHAGENT with its default settings and GPT-4o mini, the least expensive LLM; every other configuration differs from it in one respect. The *direct-prompting* baseline uses the same LLM, prompts and context, but none of the control layer: no repair, correction, retry or completion loop. The *structured-output* baseline adds to it the provider's strict JSON-schema mode, which constrains decoding to the column names, numerical types and observed categories of the real data but does not bound the number of records. The *CLLM* baseline follows direct prompting with the curation step of CLLM [10], which keeps only the records that an XGBoost model trained on the real data learns with confidence of at least 0.75 and aleatoric uncertainty of at most 0.2 (the Data-IQ criteria [58]). Eight control-layer variants change one component or repeat the reference run, and three LLM variants replace GPT-4o mini, forming a 2×2 design that crosses model generation (4o versus 4.1) with model size (mini versus full). A second provider, Anthropic, adds three LLMs under the reference configuration, Claude Haiku 4.5, Claude Sonnet 4.6 and Claude Sonnet 5.5, and repeats both baselines with Claude Haiku 5.5, used for the baselines only because its SYNTHAGENT run did not complete (Section VI-A). The Claude models run without extended reasoning and at their default temperature, which the newest of them do not allow to be changed, whereas the OpenAI models use the temperature of Table III. Token counts are compared only within a provider, because the tokenizers differ. Six sensitivity variants change one parameter of Table III at a time. Ten established generators cover the other families, all with the default settings of their reference implementations and the same divisions; the oversampling methods balance the classes, so their number of records follows the class balance, whereas the others sample N records: oversampling (SMOTE [4], ADASYN [59], Borderline-SMOTE [60]), statistical (a Gaussian copula [61] and a discrete Bayesian network learned by hill climbing [19]), tree-based (sequential CART synthesis [62] and adversarial

TABLE V
THE 36 EVALUATED CONFIGURATIONS. ALL LLM CONFIGURATIONS USE GPT-4O MINI, EXCEPT THE LLM VARIANTS AND THE BASELINES REPEATED WITH CLAUDE HAIKU 5.5. *RUN ON ALL TEN DATASETS; THE OTHERS ON THE SIX ORIGINAL ONES.

Group	Configurations	No.	Purpose
Baselines (RQ1)	Direct prompting and structured outputs, with GPT-4o mini* and Claude Haiku 5.5; CLLM*	5	Same LLM without the control layer: free or schema-constrained decoding, or a curation filter
Reference	SYNTHAGENT, default settings*	1	Point of comparison for all others
Repeated runs (RQ1)	No type rules; no post-processing; no context join; logged rerun	4	Change nothing in this setting; estimate the variation between runs
Design choices (RQ1)	Dense context; diverse context; LLM-chosen settings; range enforcement	4	Effect of context selection, of letting the LLM set ρ and the temperature, and of clipping values to the observed ranges
Sensitivity (RQ1)	ρ of 10% and 40%; b of 15 and 60; temperature 0 and 1.0	6	Effect of each fixed parameter
LLM variants (RQ4)	GPT-4.1 mini; GPT-4o; GPT-4.1*; Claude Haiku 4.5; Claude Sonnet 4.6*; Claude Sonnet 5.5	6	Effect of model generation, size and provider
Established generators* (RQ4)	SMOTE; ADASYN; Borderline-SMOTE; Gaussian copula; Bayesian network; CART; ARF; CTGAN; TVAE; TabDDPM	10	Generators fitted to the real records
Total		36	

random forests, ARF [63]) and deep (CTGAN, TVAE [5] and TabDDPM [8]). In total, the configurations give 2,800 dataset-configuration repetitions and up to 84,000 classifier fits.

Three of the control-layer variants change nothing in this setting: with no user rules, disabling post-processing removes a step that does nothing, the context join does not change the records, and disabling type rules cannot change the output because the validator never found a type violation with GPT-4o mini. Together with the reference and a rerun of it that only adds per-request logging, they are five independent runs of the same pipeline, and their spread estimates the variation between repeated runs, which we call the *noise floor*: a difference between two configurations that is smaller than this spread is not evidence of an effect.

D. Measures

Each research question has its own measures, all computed on every repetition: integrity for RQ1, fidelity and privacy for RQ2, and utility for RQ3 and RQ4, with cost reported throughout. *Integrity* is measured by the records delivered against those requested, the non-empty values, the malformed responses and the corrections. *Fidelity* compares the synthetic records alone with the real training records over the numerical features $\mathcal{J}$:

$$F_{\mathrm{JSD}} = 100\Big(1 - \tfrac{1}{|\mathcal{J}|}\textstyle\sum_{j\in\mathcal{J}} D_{\mathrm{JS}}(D_j, G_j)\Big),$$
$$F_{\mathrm{corr}} = 100\big(1 - \mathrm{mean}\,|\Sigma_D - \Sigma_G|\big), \tag{8}$$

where D_j and G_j are the values of feature j in the real and synthetic records, D_{JS} is the Jensen-Shannon divergence between their distributions, and Σ_D, Σ_G are the correlation matrices; both scores equal 100 for identical distributions. A classifier two-sample test (C2ST) [38] trains a random forest to separate real from synthetic records; an accuracy close to 0.5 means they are hard to tell apart. *Privacy* is measured by a membership-inference attack that scores each real record by its distance to the nearest synthetic record and reports the ROC AUC between training and test records, where 0.5 is chance, and by the share of *near-copies*, synthetic records within Euclidean distance 0.05 of a real record over the min-max-scaled numerical features (first 200 synthetic records of each run). *Utility* is the paired change, for metric $m \in \{\mathrm{accuracy}, \mathrm{F1}\}$, dataset d, classifier c and repetition h,

$$\Delta m_{d,c,h} = m\big(D_{d,h} \cup G_{d,h};\, c\big) - m\big(D_{d,h};\, c\big), \tag{9}$$

where $D_{d,h}$ and $G_{d,h}$ are the real training records and the synthetic records of dataset d in repetition h, and $m(\cdot; c)$ is the score of classifier c trained on them, with both models scored on the same test set. We use binary F1 for the binary tasks and class-weighted F1 for heart disease, and report changes in percentage points (pp).

E. Statistical Analysis

Repetitions and classifiers are not independent datasets, so they are averaged within each dataset and the datasets are treated as matched blocks. Configurations are compared with the Friedman test,

$$\chi_F^2 = \frac{12M}{Q(Q+1)} \sum_{a=1}^{Q} \bar{\mathrm{rk}}_a^{\,2} - 3M(Q+1), \tag{10}$$

where $\bar{\mathrm{rk}}_a$ is the average rank of configuration a over M datasets (six or ten) and Q the number of configurations, followed by two-sided Wilcoxon signed-rank tests of the best-ranked configuration against every other, with Holm correction [64], [65]. With few blocks the tests have limited power, so effect sizes and the noise floor are reported alongside p-values. As a sensitivity analysis, the comparison of LLMs is repeated with the 60 matched dataset-division pairs as blocks; these share records, so the dataset-level tests remain the primary analysis.

VI. RESULTS

This section answers the four research questions in turn, and Table VI summarizes the results. Section VI-A shows that the control layer delivers the requested dataset where the LLM alone does not (Figure 4) and that its settings change cost and fidelity but not utility (Figure 5). Section VI-B measures how closely the records follow the real data and

TABLE VI

RESULTS AT A GLANCE OVER THE TEN DATASETS: RANGES OVER THE THREE LLMS OF SYNTHAGENT, THE THREE OTHER LLM-BASED GENERATORS AND THE TEN ESTABLISHED GENERATORS. ONLY SYNTHAGENT HAS NO INTEGRITY VIOLATIONS, AND IT REACHES THE LARGEST MEDIAN F1 GAIN. BOLD: WHERE SYNTHAGENT IS BEST.

	SYNTHAGENT	Other LLM	Established
Integrity violations	**none**	13-22% missing	no LLM
Malformed responses	**repaired**	batches lost	-
Near-copies (%)	6-9	4-7	1-19
Attack AUC	0.51-0.52	0.51-0.52	0.50-0.57
Fidelity (JSD)	72-77	66-72	62-86
Median ΔF1 (pp)	-0.69 to $\mathbf{+1.43}$	-0.68 to $+0.24$	-2.15 to $+0.11$
Tokens per record	$\mathbf{-17\%}$ vs. direct	-	no LLM

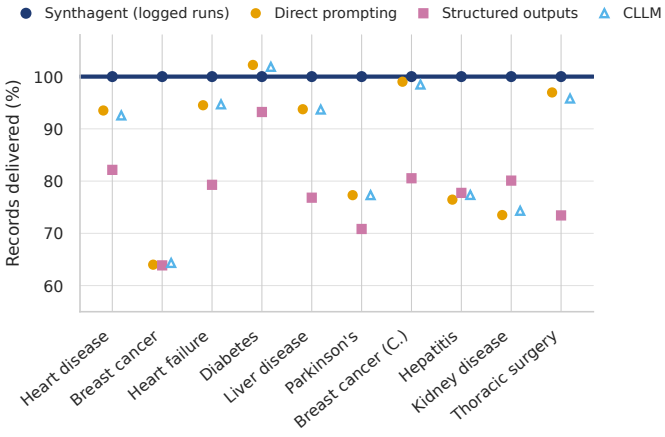


Fig. 4. Only SYNTHAGENT has no integrity violations (RQ1); the LLM baselines miss or add records on every dataset. Records delivered as a percentage of those requested, averaged over ten divisions; values above 100% are responses with more records than requested. Baselines use GPT-4o mini; SYNTHAGENT points are every run with a logged count (GPT-4o mini and GPT-4.1 on the four further datasets, Claude Sonnet 4.6 on all ten).

whether they expose real patients, Section VI-C their effect on each classifier (Figure 6), and Section VI-D compares SYNTHAGENT with the other generators and LLMs (Figure 7).

A. RQ1: Control Layer and Integrity

Finding 1. The control layer ensures integrity: without it, every run of the same LLM missed or added records; with it, no logged run had an integrity violation, at 1.9% more tokens in total and with unchanged fidelity. Figure 4 shows the records delivered per dataset. On the six original datasets, direct prompting delivered 87.6% of the configured records, fell short in 50 of 60 runs and overshot in the other 10, and lost an entire batch to each of ten malformed responses. Fidelity, privacy and utility were the same for both configurations (JSD 72.7; attack AUC 0.517 versus 0.526; F1 change -0.04 versus -0.14 pp, within the noise floor below), so the difference lies in the integrity of the dataset.

Structured outputs, the providers' option to force every response into a fixed format, do not replace the control layer (Figure 4). Every response had the declared columns and types, but the LLM returned on average 77.7% of the configured records and never the full dataset, at 21% more tokens in total than SYNTHAGENT. Forcing the format also weakened the relationships between features (correlation fidelity 63.0 versus 72.6) and made the records easier to tell from real ones (C2ST 0.943 versus 0.878), while utility did not differ (Friedman $p = .311$ for F1). A fixed format therefore guarantees the shape of each record, but not the number of records, which is what the completion loop provides. The same holds for the second provider: with Claude Haiku 5.5, direct prompting delivered 67.1% of the configured records, with 294 malformed responses, and structured outputs 82.9%. Under the control layer, the same model refused to imitate patient records in 39 of 60 repetitions; SYNTHAGENT logged each refusal, but the run finished only heart failure and is excluded.

These comparisons isolate the control layer's contribution, because the LLM, prompts and context are the same in each pair. Its contribution is integrity, at no cost in utility: with GPT-4o mini, SYNTHAGENT had no integrity violation in any logged run, while direct prompting missed 13% of the records and structured outputs 22% over ten datasets, and turned malformed responses into records instead of losing them, while the F1 change of the two did not differ (median difference $+0.08$ pp, Wilcoxon $p = .92$). On the 40 paired runs with logged counts, the difference in integrity is significant (exact McNemar test, $p < 10^{-11}$), and SYNTHAGENT needed 17% fewer tokens per delivered record than direct prompting and 40% fewer than structured outputs (Wilcoxon $p < 10^{-6}$), because the baselines pay for records they lose. The results therefore come from the two together: integrity is due to the control layer, which achieved it with every LLM whose run completed while no LLM achieved it alone, and the utility gains come from the LLM's knowledge, which the control layer turns into a usable dataset (Section VI-D).

The design choices inside the control layer matter much less than the control layer itself (Figure 5). The five repeated runs of the same pipeline differ by up to 0.5 JSD points, 2 correlation points and 0.7 pp of utility, which is the noise floor. Dense context stays inside it; diverse context slightly improves fidelity at a slightly higher attack AUC (0.544); LLM-chosen settings cost 57% more tokens; and range enforcement lowers JSD fidelity by 0.9 points. The Friedman tests find no difference in utility between the variants ($p = .843$ for accuracy, $p = .872$ for F1). The parameters of Table III behave the same way: changing the context sample, the batch size or the temperature leaves utility within the noise floor ($p = .986$ accuracy, $p = .178$ F1), but it changes cost and fidelity. Halving the context sample saves 27% of the tokens and doubling the batch size 39%, whereas smaller batches cost 73% more. A temperature of 1.0 raises JSD fidelity by 3.2 points and correlation fidelity by 8.7 points (C2ST 0.799 against 0.878) at no cost in utility, privacy or tokens, so the default of 0.3 is conservative for fidelity.

The corrections make a less disciplined LLM usable. GPT-4o mini produced no type violation, and the local parser fixed every defect in its responses. GPT-4.1 mini, the fastest LLM, produced on average 7.4 type violations per repetition on

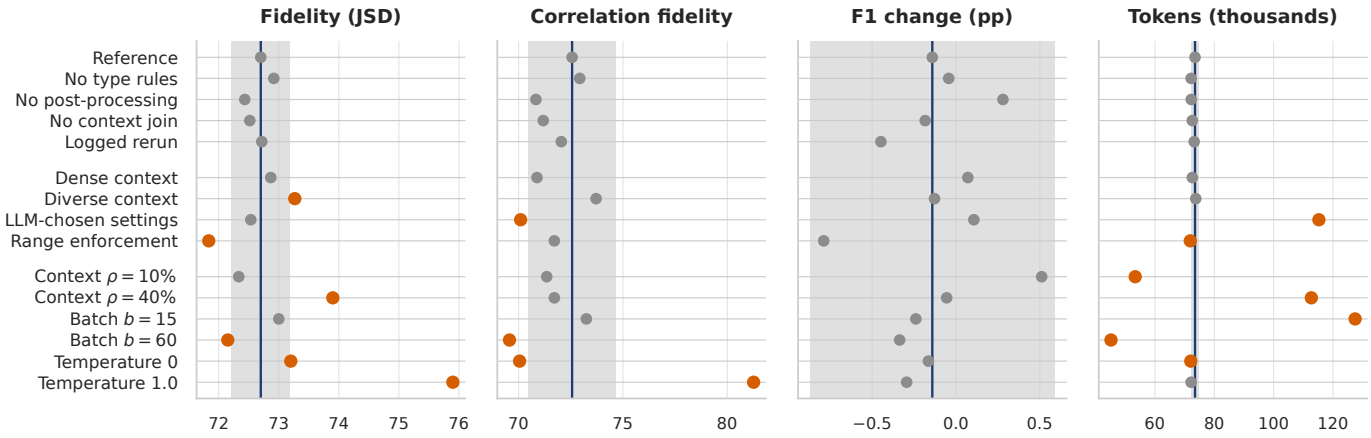


Fig. 5. No design choice or parameter moves utility beyond the noise floor; the parameters change only cost and fidelity (RQ1). GPT-4o mini, six original datasets. The first five rows rerun the same pipeline; grey band: noise floor (reference, blue line, ± their spread); orange: outside it.

breast cancer and one malformed response; these were sent back for correction instead of entering the dataset, and its datasets had no integrity violations and gave the best accuracy change of the OpenAI LLMs (Section VI-D).

B. RQ2: Fidelity and Privacy

Finding 2. The generated records follow the real distributions reasonably well without revealing the real records, but they remain distinguishable from them (Table VII). Fidelity averages 72.7 out of 100, from 65.2 on heart disease, the furthest below its required sample size, to 80.2 on diabetes, the closest. The membership-inference attack stays near chance (AUC 0.526 on average, at most 0.567), which is a diagnostic for one attack rather than a formal guarantee. The records are also not interpolations of training patients: 7.8% are near-copies, against 17-19% for the oversampling methods and comparable to the 1-8% of the statistical, tree-based and deep generators. Nor do the LLMs recall the public datasets: near-copies of held-out test records, never shown to the LLM, are rarer (2.6-4.1%) than of an equal-sized training sample (3.7-5.0%), as for the fitted generators. A classifier can still separate real from synthetic records (C2ST 0.878), so synthetic records complement real ones rather than replace them.

C. RQ3: Utility for Classifiers Trained on Small Data

Finding 3. The generated records help the classifiers that fail on the minority class and slightly harm those that are already strong, so their effect depends on the dataset and the classifier. On heart failure, for example, SVM and kNN reach over 60% accuracy with an F1 of almost zero because they rarely predict the minority class. With the reference configuration, F1 improves in 26 of the 59 classifier-dataset combinations and decreases in 31 (Figure 6); the largest gains occur where real-only training fails, such as kNN on heart failure (+15.3 F1 points). No combination is significant after Holm correction. Trained on synthetic records alone, classifiers keep a median 85-98% of their real-only F1, close to the statistical and tree-based generators (92-99%). On average, F1 is unchanged with GPT-4o mini (−0.14 pp) and gains more with stronger LLMs (Section VI-D).

TABLE VII
THE SYNTHETIC RECORDS FOLLOW THE REAL DISTRIBUTIONS AND KEEP THE ATTACK NEAR CHANCE ON EVERY DATASET (RQ2, RQ3). REFERENCE CONFIGURATION (GPT-4O MINI) PER ORIGINAL DATASET, OVER TEN REPETITIONS AND ALL CLASSIFIERS; JSD AND CORRELATION ON A 0-100 SCALE.

	Fidelity and privacy				Acc. (%)		Change (pp)	
Dataset	JSD	Corr.	C2ST	AUC	Real	Aug.	Acc.	F1
Breast	77.2	74.5	0.911	0.550	95.1	94.4	−0.66	−0.62
Diabetes	80.2	44.0	0.897	0.504	74.2	74.1	−0.03	+2.39
Heart dis.	65.2	76.6	0.893	0.507	56.6	53.4	−3.15	−0.97
Heart fail.	71.2	81.2	0.892	0.534	74.1	71.6	−2.53	−1.00
Liver	71.8	73.8	0.883	0.495	67.4	67.6	+0.19	+0.13
Parkinson's	70.6	85.3	0.789	0.567	84.9	84.0	−0.97	−0.77
Mean	72.7	72.6	0.878	0.526	75.4	74.2	−1.19	−0.14

TABLE VIII
UNDER THE CONTROL LAYER, NO LLM PRODUCED AN INTEGRITY VIOLATION IN ANY LOGGED RUN; THE CLAUDE MODELS GIVE THE HIGHEST FIDELITY AND F1 GAIN (RQ4). REFERENCE CONFIGURATION; SECONDS AND TOKENS ARE MEANS PER REPETITION, TOKENS COMPARABLE ONLY WITHIN A PROVIDER.

LLM	JSD	Corr.	ΔAcc.	ΔF1	Sec.	Tokens
GPT-4o mini	72.70	**72.57**	−1.19	−0.14	378.7	73,510
GPT-4.1 mini	73.60	62.82	−0.23	+0.92	**168.5**	**72,069**
GPT-4o	73.95	64.32	−0.50	+0.25	255.2	80,046
GPT-4.1	75.72	63.30	−0.44	+1.09	201.7	80,652
Claude Haiku 4.5	74.85	69.88	−0.34	+1.55	253.5	78,020
Claude Sonnet 4.6	**79.07**	70.77	**+0.40**	**+2.39**	373.5	**75,504**
Claude Sonnet 5.5	79.00	63.78	+0.10	+1.62	205.5	86,074

D. RQ4: Comparison with Established Generators and LLMs

Finding 4. SYNTHAGENT achieves the largest median F1 gain among the generators compared, and the LLM is the setting that most changes the results. Among the LLMs (Table VIII), the Claude models give the strongest results in

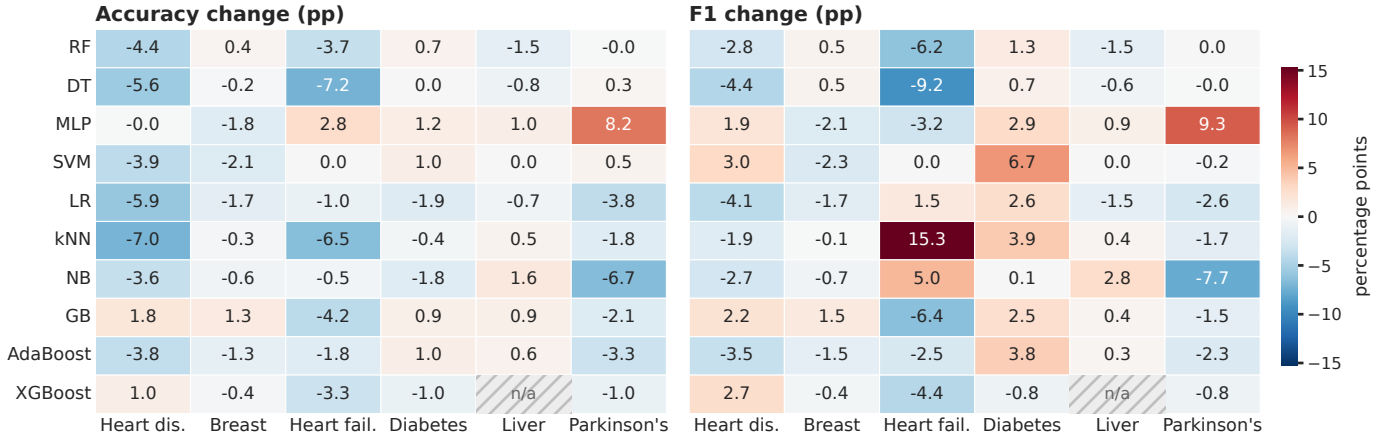


Fig. 6. The synthetic records help the classifiers that fail on real data alone and slightly harm strong ones (RQ3). Change from real-only to real-plus-synthetic training, reference configuration, per classifier and dataset; hatched: not evaluated.

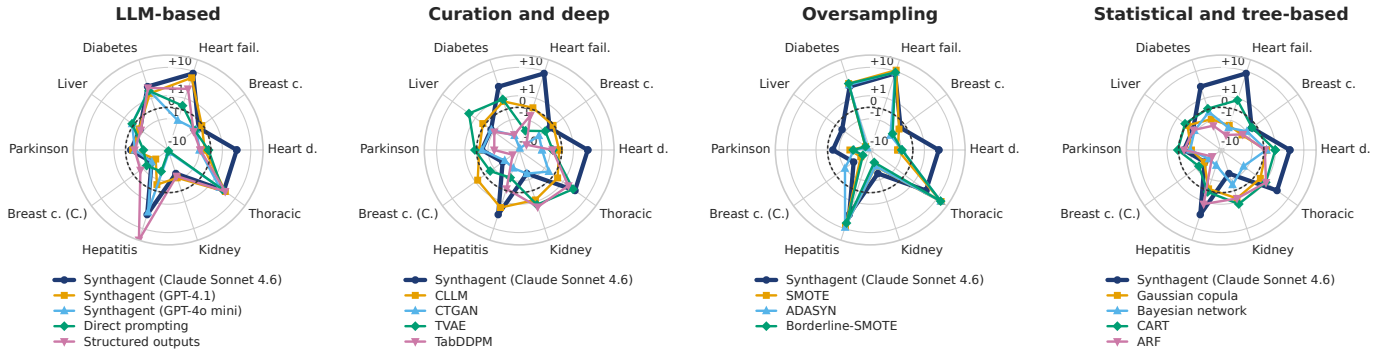


Fig. 7. SYNTHAGENT gains most where real-only training is weak; oversampling swings most (RQ4). F1 change (pp) per dataset for every generator, one panel per group, each against SYNTHAGENT with Claude Sonnet 4.6 (dark blue; the name in parentheses is the LLM that SYNTHAGENT uses); symmetric logarithmic radius (linear within ±1 pp); dashed circle: no change.

our runs: Claude Sonnet 4.6 reaches the highest fidelity (79.07) and the largest gains (+0.40 pp of accuracy, +2.39 pp of F1), and all three Claude models exceed the best OpenAI model in F1. This comparison is suggestive rather than conclusive, since the Claude models ran at their default temperature and the OpenAI models at 0.3. At list prices, a synthetic dataset costs $0.02 (GPT-4o mini) to $0.49 (Claude Sonnet 4.6). Within OpenAI, GPT-4.1 gives the highest fidelity (75.72) and F1 gain (+1.09 pp), while the three stronger OpenAI models preserve correlations less well (62.8 to 64.3, against 72.6 for GPT-4o mini). The control layer delivered every dataset with every LLM, and the differences between them are not significant across six datasets ($p = .572$ and $p = .615$ within OpenAI, $p = .377$ over all seven, for accuracy and F1). With the 60 divisions as blocks, the seven LLMs differ ($p \leq .002$), and Claude Sonnet 4.6 gives a higher F1 than each of the others after Holm correction ($p \leq .039$).

Over the ten datasets (Table VI, Figure 7), no generator is best on every measure. The tree-based and statistical generators reproduce the real distributions most closely (JSD 85.0 to 86.0), and CART's records are as hard to tell from real ones as real records are (C2ST 0.465), but have the highest attack AUC (0.565). High fidelity does not imply utility: the Bayesian network matches the copula in fidelity (JSD 85.1) but improves F1 on only one dataset. Oversampling lowers accuracy (median −2.3 to −3.1 pp) and varies the most in F1 (SD above 7 pp). SYNTHAGENT with Claude Sonnet 4.6 gives the largest median F1 gain of all sixteen generators (+1.43 pp) and improves F1 on half of the datasets, more often than any other LLM configuration except CLLM. The CLLM filter changes the data least: it gives the best median accuracy change (+0.07 pp) and the smallest variation in F1, and improves F1 on 70% of the datasets, and CART on 80%, both with small gains (+0.24 and +0.11 pp); but CLLM missed 13% of the requested records, the shortfall of direct prompting, whose output it filters. The generators differ across the ten datasets (Friedman $p < .001$ for accuracy, $p = .004$ for F1), with SYNTHAGENT and Claude Sonnet 4.6 ranked first in F1 and CLLM in accuracy; after Holm correction, CLLM has a higher accuracy than SMOTE and Borderline-SMOTE, and SYNTHAGENT with Sonnet 4.6 a higher accuracy than Borderline-SMOTE, and a higher F1 than TabDDPM on every dataset ($p = .029$).

VII. Discussion and Limitations

We return to the question of Section I: whether SYNTHAGENT makes LLM-based SDG deliver datasets of assured integrity that are useful for small healthcare data. The results come from the LLM and the control layer together: the LLM brings domain knowledge, and the control layer turns it into a dataset that satisfies its integrity constraints, which no LLM did alone, at almost no extra cost (RQ1). The usefulness is real but bounded (RQ2-RQ4). SYNTHAGENT is not the best choice in three situations. With a weak LLM it does not improve utility over direct prompting, and on kidney disease GPT-4o mini lost 10.3 pp of F1 with or without it. When the goal is the closest fit to the real distribution, tree-based and statistical generators are better, at a higher membership-attack risk. When stability matters more than gain, the CLLM filter changes the data least, at the price of missing records; combining it with the control layer is a natural extension. No generator improves every dataset: SDG cannot replace missing patients [27, Sec. 2.8]. In practice, the LLM matters most, and synthetic records should be checked per classifier.

The study has several limitations. Ten datasets (six for the variants) limit the power of the tests, and the divisions are not stratified. The datasets are public, so part of the fidelity may reflect records memorized by the LLMs, which the attack does not test; private data are left for future work. Parkinson's recordings of one subject can fall in both sets. Columns and types are inferred from the full dataset. The parameters in Table III were fixed and not tuned; the sensitivity study varies three of them one at a time, not jointly. The temperature differs between providers, and part of the Claude models' higher fidelity may reflect it (Section VI-D). Privacy is measured with one attack, not enforced, and no result establishes differential privacy [42]. Delivered counts were not logged for the GPT runs of SYNTHAGENT on the six original datasets. The CLLM baseline reimplements its curation step rather than running the original code. The corrections were exercised only by GPT-4.1 mini, and records obtained in the last correction round or in the completion loop are not validated again. Context selection is not class-stratified, so a rare class can be missing from a prompt. Local LLMs were not evaluated, and Claude Haiku 5.5's refusals show that a provider's safety behaviour can block generation. The established generators use default settings, and GReaT, which needs a GPU, was not included.

Synthetic records must not inform diagnosis without domain review, nor be released on the strength of one attack. Everything needed to reproduce the experiments is publicly available: the implementation, the prompts, the configuration and LLM identifier of every run, the dataset preparation, pinned dependencies, the per-run results with the synthetic records saved during the runs, and the scripts that regenerate every table and figure.[1]

[1]https://github.com/smartlord7/Synthagent

VIII. Conclusion

Generators fitted to small healthcare datasets can only recombine their patterns, whereas an LLM adds domain knowledge but returns text rather than a dataset. SYNTHAGENT resolves this with a control layer that turns LLM responses into synthetic datasets without integrity violations, with a recorded cost, without training on the target data. On ten healthcare datasets, direct prompting violated integrity in every run, while SYNTHAGENT never did, with fewer tokens per delivered record, and it achieved the largest median F1 gain of sixteen generators. Future work will evaluate local LLMs, class-aware context, privacy filters and the deployment of Section IV.

Acknowledgment

This work is funded by national funds through FCT - Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 - Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025.

AI-Generated Content Acknowledgement

The authors used Claude (Anthropic) to assist with code development and with the writing of the manuscript, and take full responsibility for all content.

References

[1] N. Rieke, J. Hancox, W. Li, F. Milletarì, H. R. Roth, S. Albarqouni, S. Bakas, M. N. Galtier, B. A. Landman, K. Maier-Hein, S. Ourselin, M. Sheller, R. M. Summers, A. Trask, D. Xu, M. Baust, and M. J. Cardoso, "The future of digital health with federated learning," *npj Digital Medicine*, vol. 3, no. 1, p. 119, 2020.

[2] R. D. Riley, K. I. E. Snell, J. Ensor, D. L. Burke, F. E. Harrell, Jr., K. G. M. Moons, and G. S. Collins, "Minimum sample size for developing a multivariable prediction model: Part II – binary and time-to-event outcomes," *Statistics in Medicine*, vol. 38, no. 7, pp. 1276–1296, 2019.

[3] Z. Qian, R. Davis, and M. van der Schaar, "Synthcity: A benchmark framework for diverse use cases of tabular synthetic data," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[5] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[6] V. Borisov, K. Seßler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=cEygmQNOeI

[7] A. V. Solatorio and O. Dupriez, "REaLTabFormer: Generating realistic relational and tabular data using transformers," *arXiv preprint arXiv:2302.02041*, 2023.

[8] A. Kotelnikov, D. Baranchuk, I. Rubachev, and A. Babenko, "TabDDPM: Modelling tabular data with diffusion models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 17 564–17 579.

[9] H. Zhang, J. Zhang, Z. Shen, B. Srinivasan, X. Qin, C. Faloutsos, H. Rangwala, and G. Karypis, "Mixed-type tabular data synthesis with score-based diffusion in latent space," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=4Ay23yeuz0

[10] N. Seedat, N. Huynh, B. van Breugel, and M. van der Schaar, "Curated LLM: Synergy of LLMs and data curation for tabular augmentation in low-data regimes," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024.

[11] C. Ge, S. Mohapatra, X. He, and I. F. Ilyas, "Kamino: Constraint-aware differentially private data synthesis," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1886–1899, 2021.

[12] J. Rorseth, P. Godfrey, L. Golab, D. Srivastava, and J. Szlichta, "Recovering structure in unstructured LLM outputs," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 4329–4330.

[13] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[14] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proceedings of the 2nd SysML Conference*, 2019.

[15] J. Kim, T. Kim, and J. Choo, "EPIC: Effective prompting for imbalanced-class data synthesis in tabular data classification via large language models," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[16] R. Y. Wang and D. M. Strong, "Beyond accuracy: What data quality means to data consumers," *Journal of Management Information Systems*, vol. 12, no. 4, pp. 5–33, 1996.

[17] C. Batini, C. Cappiello, C. Francalanci, and A. Maurino, "Methodologies for data quality assessment and improvement," *ACM Computing Surveys*, vol. 41, no. 3, pp. 16:1–16:52, 2009.

[18] A. D. Lautrup, T. Hyrup, A. Zimek, and P. Schneider-Kamp, "SynthEval: A framework for detailed utility and privacy evaluation of tabular synthetic data," *Data Mining and Knowledge Discovery*, vol. 39, no. 1, p. 6, 2025.

[19] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2014.

[20] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[21] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 10 932–10 952.

[22] K. Cai, X. Lei, J. Wei, and X. Xiao, "Data synthesis via differentially private Markov random fields," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2190–2202, 2021.

[23] C. Lee, J. Kim, and N. Park, "CoDi: Co-evolving contrastive diffusion models for mixed-type tabular synthesis," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 18 940–18 956.

[24] J. Fan, T. Liu, G. Li, J. Chen, Y. Shen, and X. Du, "Relational data synthesis using generative adversarial networks: A design space exploration," *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1962–1975, 2020.

[25] A. Shankar, H. Brouwer, R. Hai, and L. Y. Chen, "SiloFuse: Cross-silo synthetic data generation with latent tabular diffusion models," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 110–123.

[26] L. Chen, C. Wu, J. Li, S. Liu, G. Ye, and H. Chai, "TabLoft: Tabular data generation based on LLM with ordered features," in *Proceedings of the IEEE 42nd International Conference on Data Engineering (ICDE)*, 2026, pp. 1044–1056.

[27] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006.

[28] J. Hyeong, J. Kim, N. Park, and S. Jajodia, "An empirical study on the membership inference attack against tabular data synthesis models," in *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM)*, 2022, pp. 4064–4068.

[29] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *Proceedings of the IEEE 29th International Conference on Data Engineering (ICDE)*, 2013, pp. 458–469.

[30] J. Qin, S. Huang, Y. Wang, J. Zhu, Y. Zhang, Y. Miao, R. Mao, M. Onizuka, and C. Xiao, "BClean: A Bayesian data cleaning system," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 3407–3420.

[31] A. Narayan, I. Chami, L. Orr, S. Arora, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[32] W. Ni, K. Zhang, X. Miao, X. Zhao, Y. Wu, Y. Wang, and J. Yin, "ZeroED: Hybrid zero-shot error detection through large language model reasoning," in *Proceedings of the IEEE 41st International Conference on Data Engineering (ICDE)*, 2025, pp. 3126–3139.

[33] S. Shankar, T. Chambers, T. Shah, A. G. Parameswaran, and E. Wu, "DocETL: Agentic query rewriting and evaluation for complex document processing," *Proceedings of the VLDB Endowment*, vol. 18, no. 9, pp. 3035–3048, 2025.

[34] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.

[35] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[36] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proceedings of the IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 13–24.

[37] S. Guha, F. A. Khan, J. Stoyanovich, and S. Schelter, "Automated data cleaning can hurt fairness in machine learning-based decision making," in *Proceedings of the IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 3747–3754.

[38] D. Lopez-Paz and M. Oquab, "Revisiting classifier two-sample tests," in *International Conference on Learning Representations*, 2017.

[39] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18.

[40] A. Machanavajjhala, J. Gehrke, D. Kifer, and M. Venkitasubramaniam, "ℓ-diversity: Privacy beyond k-anonymity," in *Proceedings of the 22nd International Conference on Data Engineering (ICDE)*, 2006, p. 24.

[41] N. Li, T. Li, and S. Venkatasubramanian, "t-closeness: Privacy beyond k-anonymity and ℓ-diversity," in *Proceedings of the IEEE 23rd International Conference on Data Engineering (ICDE)*, 2007, pp. 106–115.

[42] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, ser. Foundations and Trends in Theoretical Computer Science. Now Publishers, 2014, vol. 9, no. 3–4.

[43] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: Facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.

[44] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Ag"uera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[45] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[46] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.

[47] Q. Chen, Y. Chen, B. Yan, X. Jiang, X. Zhang, Y. Kang, T. Zhang, W. Huang, C. Gao, L. Fan, and Q. Yang, "Model trip: Enhancing privacy and fairness in model fusion across multi-federations for trustworthy global healthcare," in *Proceedings of the IEEE 40th International Conference on Data Engineering (ICDE)*, 2024, pp. 2231–2244.

[48] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart disease," UCI Machine Learning Repository, 1989.

[49] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast cancer wisconsin (diagnostic)," UCI Machine Learning Repository, 1993.

[50] "Heart failure clinical records," UCI Machine Learning Repository, 2020.

[51] B. Ramana and N. Venkateswarlu, "ILPD (indian liver patient dataset)," UCI Machine Learning Repository, 2022.

[52] M. Little, "Parkinsons," UCI Machine Learning Repository, 2007.

[53] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 1988, pp. 261–265.

[54] M. Patrício, J. Pereira, J. Crisóstomo, P. Matafome, M. Gomes, R. Seiça, and F. Caramelo, "Using resistin, glucose, age and BMI to predict the presence of breast cancer," *BMC Cancer*, vol. 18, p. 29, 2018.

[55] "Hepatitis," UCI Machine Learning Repository, 1983.
[56] L. Rubini, P. Soundarapandian, and P. Eswaran, "Chronic kidney disease," UCI Machine Learning Repository, 2015.
[57] M. Zieba, J. M. Tomczak, M. Lubicz, and J. Świątek, "Boosted SVM for extracting rules from imbalanced data in application to prediction of the post-operative life expectancy in the lung cancer patients," *Applied Soft Computing*, vol. 14, pp. 99–108, 2014.
[58] N. Seedat, J. Crabbé, I. Bica, and M. van der Schaar, "Data-IQ: Characterizing subgroups with heterogeneous outcomes in tabular data," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
[59] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proceedings of the IEEE International Joint Conference on Neural Networks (IJCNN)*, 2008, pp. 1322–1328.
[60] H. Han, W.-Y. Wang, and B.-H. Mao, "Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning," in *Advances in Intelligent Computing (ICIC)*, ser. Lecture Notes in Computer Science, vol. 3644, 2005, pp. 878–887.
[61] N. Patki, R. Wedge, and K. Veeramachaneni, "The synthetic data vault," in *Proceedings of the IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2016, pp. 399–410.
[62] B. Nowok, G. M. Raab, and C. Dibben, "synthpop: Bespoke creation of synthetic data in R," *Journal of Statistical Software*, vol. 74, no. 11, pp. 1–26, 2016.
[63] D. S. Watson, K. Blesch, J. Kapar, and M. N. Wright, "Adversarial random forests for density estimation and generative modeling," in *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 206, 2023, pp. 5357–5375.
[64] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.
[65] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.